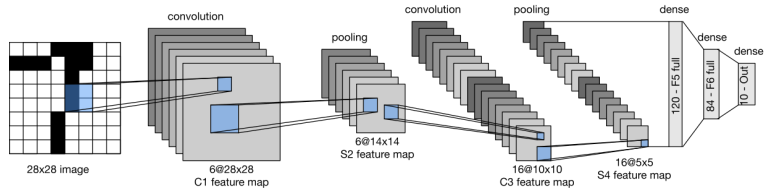


Sprzętowa implementacja sieci neuronowej CNN w układzie FPGA do rozpoznawania cyfr pisanych odręcznie

Michał Szulejko

Promotor: dr inż. Robert Strąkowski

Architektura LeNet-5



LeNet-5 - Pierwsza sieć konwolucyjna (Yann LeCun, lata 90.)

Cztery kluczowe pomysły:

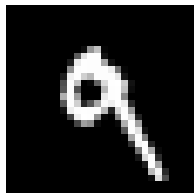
- 1 **Kernele** - małe filtry szukające wzorów
- 2 **Współdzielenie wag** - mniej pamięci
- 3 **Pooling** - zmniejszanie rozmiaru, zachowanie cech
- 4 **Warstwowe uczenie** - każda warstwa uczy się bardziej skomplikowanych wzorów

Architektura:

- Conv1: 6 filtrów, 5×5 kernel $\rightarrow$ 117,600 MAC
- Conv2: 16 filtrów, 5×5 kernel $\rightarrow$ 240,000 MAC
- FC1: $400 \rightarrow 120$ neuronów $\rightarrow$ 48,000 MAC
- FC2: $120 \rightarrow 84$ neuronów $\rightarrow$ 10,080 MAC
- FC3: $84 \rightarrow 10$ neuronów $\rightarrow$ 840 MAC
- Razem: 61,706 parametrów, 416,520 MAC

Założenia MNIST:

- Wymiary: 28×28 pikseli
- Typ: Czarno-białe
- Wartość piksela: 0-255 (czarny do białego)
- Normalizacja: Każdy piksel dzielony przez 255 $\rightarrow [0.0, 1.0]$
- Zestaw treningowy: 60,000 obrazów
- Zestaw testowy: 10,000 obrazów
- Każdy obraz ma etykietę (cyfra 0-9)



Przykład cyfry: 9

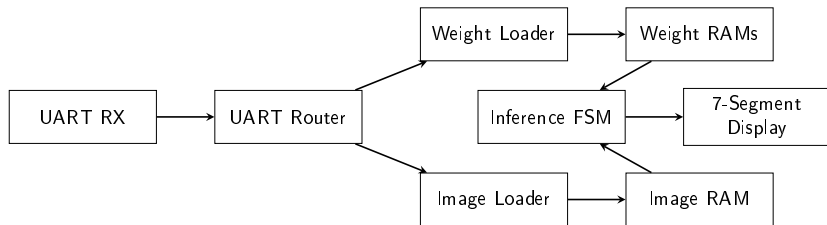
Krok 1: Trening w PyTorch

- Załaduj MNIST (60,000 obrazów treningowych)
- Trening sieci przez 10 epok
- Użycie optymalizatora Adam
- Loss function: Cross-Entropy
- Rezultat: **98.9% dokładności** na testach

Krok 2: Post-Training Quantization (PTQ)

- Problem: Float32 wagi zajmują dużo pamięci (~ 246 KB), obliczenia trwają dłużej oraz potrzebują więcej zasobów układu
- Rozwiązanie: Konwersja na Int8 (~ 61 KB)
- Oszczędność: **4x mniej pamięci!**
- Spadek dokładności: 98.9% $\rightarrow$ **97.6%** (zaledwie 1.3%)

System inferencji FPGA



Przepływ danych: PC → UART → FPGA

Główne moduły Verilog:

- 1 **UART Router** - odbiera dane, rozpoznaje znaczniki
- 2 **Weight Loader** - parsuje wagi (62,670 bajtów)
- 3 **Image Loader** - parsuje obraz (784 bajty)
- 4 **Inference FSM** - 44 stanów: Conv1 → Pool1 → Conv2 → Pool2 → FC1 → FC2 → FC3

Organizacja Pamięci:

- Weight RAM (Conv + FC BRAM + Tanh LUT): ~61 KB
- Image RAM: 0.8 KB
- Intermediate Buffers (A/B/C): ~6.5 KB
- Razem: ~68 KB

Połączenie Fizyczne

- Baudrate: 115,200 bps
- Transfer wag (62,670 B)
- Transfer obrazu (784 B)

Protokół ze Znacznikami:

- Wagi: Start (0xAA 0x55) → Dane (62,670 B) → End (0x55 0xAA)
- Obraz: Start (0xBB 0x66) → Dane (784 B) → End (0x66 0xBB)

Dlaczego Znaczniki?

- 1 Synchronizacja - informacja dla systemu, kiedy rozpocząć transmisję
- 2 Bezpieczeństwo - potwierdzenie zakończenia transmisji sukcesem
- 3 Rozpoznanie przesyłanych danych - wagi lub obraz

- **Przygotowanie danych** - stworzenie referencyjnych logitów za pomocą Pythona
- **Testbench** - testowanie izolowanego modułu inferencji
- **Test end-to-end** - pełny test porównania predykcji i logitów z uwzględnieniem wszystkich modułów

Testowanie na MNIST:

Wersja	Dokładność	Testy
PyTorch (FLOAT32)	98.9%	9,500
Python (INT8)	97.6%	9,500
Verilog (INT8)	97.6%	9,500
FPGA (INT8)	97.6%	9,500

Predykcje FPGA są zgodne z symulacją w Pythonie w 100%

Wdrożyłem historyczną sieć **LeNet-5** na **FPGA** z:

- **97.6%** dokładności (spadek 1.3% względem FP32)
- Zgodność logitów w symulacji Pythonowej oraz inferencji na **FPGA**
- **~68 KB** pamięci

Dzięki **kwantyzacji**, **optymalizacji w Verilog'u** i **rozproszonej pamięci RAM** mogę uruchamiać sieci na małych, tanich urządzeniach.

To pokazuje, że nie potrzebujemy GPU do inteligentnych systemów wbudowanych.

Dziękuję za uwagę

Michał Szulejko